Arcade : Developers documentation

Hello! Here you will find the documentation to help you implement your own games and graphics libraries in Arcade.

First, we will explain how to add a game to Arcade, and then we will walk you through implementing your own graphics libraries.

I. How does Arcade works

The goal of Arcade is to allow the same game to run using different graphics libraries. To achieve this, the program is divided into 3 parts:

- The "graphics" part, which handles rendering sprites at the correct positions.
- The "game" part, which handles calculating interactions between entities and their positions (relative to the game).
- The "core" part, which links the "graphics" and "game" sections and notably contains the main game loop.

You do not need to modify the "core" part to implement new games and/or new graphics libraries.

To make the project architecture clearer, we have provided an UML diagram below containing all our interfaces and classes, helping you better understand how they interact with one another.

To compile the project using our CMake lists, you need to add a few lines to the CMake list.

Let’s say you’re developing a new game for Arcade. Go to the “games” folder at the root of the repository, then open the CMakelists.txt file.

Next, add these lines above the “add_custom_targets” rule:

```
 set(Name
    folder_name
)
set(SRCS_GameName
    ${Name}/src/FileName.cpp
)

add_library(${Name} SHARED ${SRCS_Name})
target_include_directories(${Name} PRIVATE ${SHARED} ${Name}/include/)
target_link_libraries(${Name} sfml-graphics sfml-window sfml-system sfml-audio)
set_target_properties(${Name} PROPERTIES PREFIX "")
```

DisplayNcurses.hpp | CMakeLists.txt | DisplayNcurses.cpp | IEntity.hpp | IDisp

```
set(SRCS_NCURSES
        ${NCURSES}/src/DisplayNcurses.cpp
        arcade_ncurses/src/DisplayNcurses.cpp
)

set(SHARED
        ../../include/Shared
)

add_library(${NCURSES} SHARED ${SRCS_NCURSES})

target_include_directories(${NCURSES} PRIVATE ${SHARED} ${NCURSES}/include/)
target_link_libraries(${NCURSES} ncurses)
set_target_properties(${NCURSES} PROPERTIES PREFIX "")

set(SFML
    arcade_sfml
)

set(SRCS_SFML
    ${SFML}/src/DisplaySFML.cpp
)

add_library(${SFML} SHARED ${SRCS_SFML})
target_include_directories(${SFML} PRIVATE ${SHARED} ${SFML}/include/)
target_link_libraries(${SFML} sfml-graphics sfml-window sfml-system sfml-audio)
set_target_properties(${SFML} PROPERTIES PREFIX "")

add_custom_target(move ALL
    DEPENDS ${SDL2} ${NCURSES} ${SFML}
    COMMAND mv *.so ../
)
```

Here, we added the SFML graphical lib to the CMakelist.txt

Then, modify “add_custom_target” :

add_custom_target(move ALL

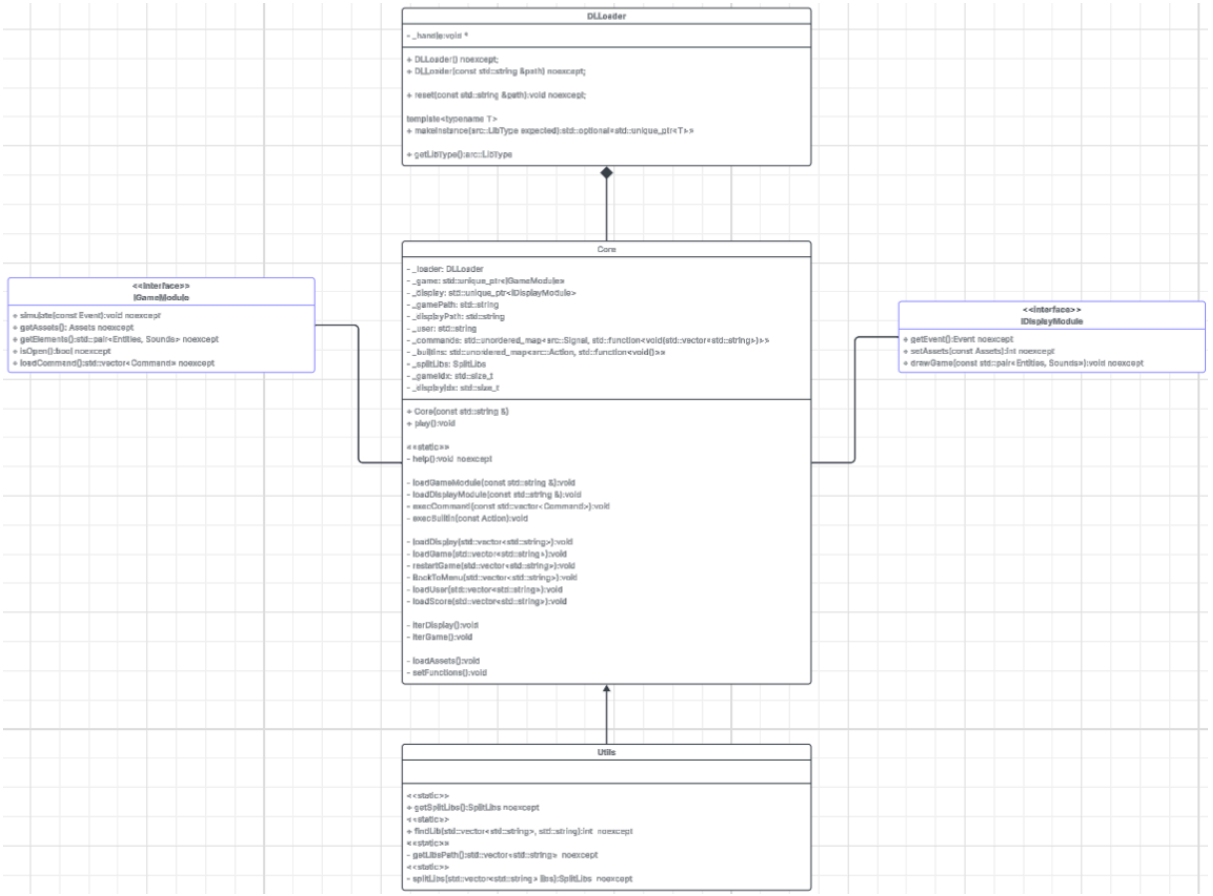
DEPENDS ${SDL2} ${NCURSES} ${*Name*}

COMMAND mv *.so ../

```
add_library(${SFML} SHARED ${SRCS_SFML})
target_include_directories(${SFML} PRIVATE ${SHARED} ${SFML}/include/)
target_link_libraries(${SFML} sfml-graphics sfml-window sfml-system sfml-audio)
set_target_properties(${SFML} PROPERTIES PREFIX "")

add_custom_target(move ALL
    DEPENDS ${SDL2} ${NCURSES} ${SFML}
    COMMAND mv *.so ../
)
```

Here is the UML graph to understand our codebase.



II. How to implement a new game

Now that you know how Arcade works as a whole, we will see how to implement new features. First, I will explain how to create a game compatible with our "arcade machine."

As seen previously, the first step is to create files in the game folder at the root of the repo and add information to the CMakeLists.txt. Once that is done, you will be able to develop your game.

First, create a class that will inherit from "public IGameModule". Next, implement all the functions of the "IGameModule.hpp" interface.

You have an UML diagram at the end of this section to explain how IEntities and ISound works.

A. Simulate()

The purpose of the simulate method is to calculate the new positions of all entities and determine their state.

This function literally simulates the game (it allows you to increment the score, move your character, kill it if it is dead, etc.).

It takes an Event as a parameter, which is a pair composed of an "Action" and a "Vector2f", corresponding respectively to the key used (keyboard key, mouse click...) and the position of the mouse at the time of the "Action" (a Vector2f is a vector of floats).

Thus, this function allows you to calculate the new state of your game based on the key pressed by the user and the position of the mouse.

B. getAssets()

The getAssets method is a getter which returns an Assets. An Assets is a pair of std::string vectors, which contains respectively the names of the graphics assets and the names of the songs assets.

This function is useful only in core.cpp because it transmits the name of the assets to display to the graphic library.

If you need examples, feel free to take a look at GameSnake.cpp.

C. getElements()

This function returns a pair of "Entities" and "Sounds". "Entities" is a vector of "IEntity", which contains all the information relative to an entity: its position, size, color, etc.

This information is used to display each entity during "Display". "Sounds" is a vector of "ISound", which contains all the sounds to be played during the game's execution.

This function is only useful in "core.cpp", which transmits this information to the graphical part of the project. You will normally not need to use it.

D. isOpen()

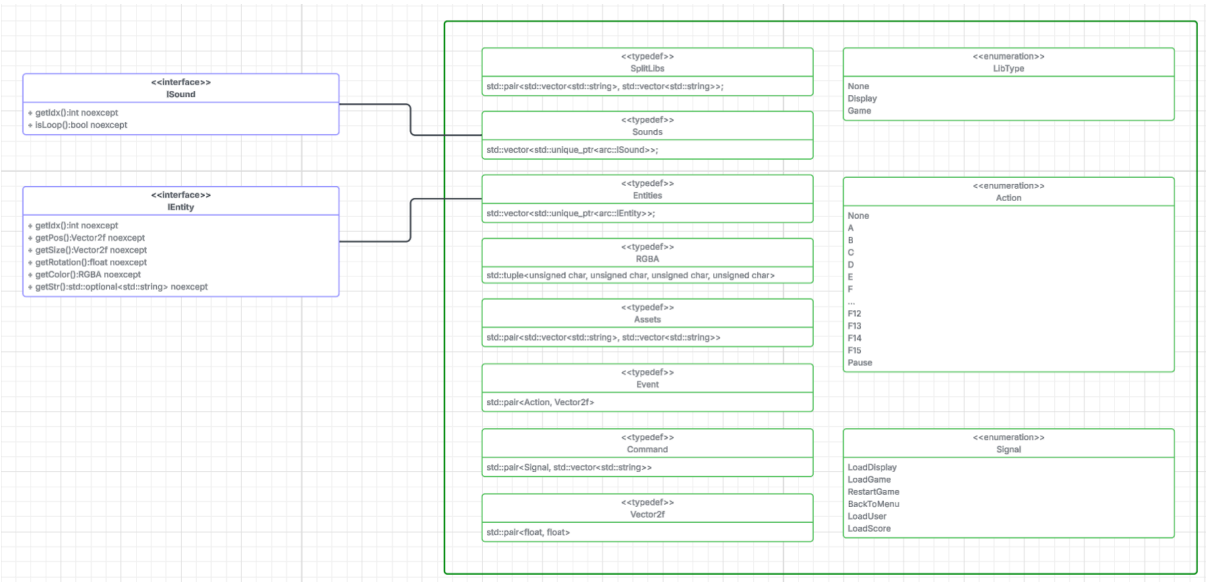
This function returns true if a window is open, otherwise it returns false.

C. loadCommand()

This function returns a "Command". "Command" corresponds to the action the core needs to take; for example, it tells the core whether to restart the game, display the menu, load a different game, and so on.

This function is used only by the core, so you will not need to use it yourself.

All of these functions must be implemented if you want your game to be suitable with our Arcade program. Feel free to add any other functions needed by your game.



III. How to implement a graphic lib

Perhaps you want to improve Arcade by implementing a new graphical library for this magnificent arcade machine! The following section will show you how to add a new graphical library to the arcade machine. If you feel lost regarding the project's architecture, do not hesitate to look at the UML diagrams provided previously.

A. getEvent()

The purpose of the "getEvent" function is to retrieve the key used (keyboard key, mouse click...), and the mouse position.

This function returns an "Event", which is composed of an "Action" and a "Vector2f".

The Action corresponds to the key pressed by the player; therefore, the key from the graphical library must be "translated" into an Action. As for the "vector2f", it stores the position of the mouse as a pair of coordinates (x, y).

B. setAssets()

The "setAssets" function takes an "Asset" as a parameter, which consists of a pair of std::string vectors.

The first element of the pair contains the path to the graphical assets without their extension, which you must add manually.

The other element of the pair contains the path to the audio files, including their extension.

The "setAssets" function is used to open the assets only once throughout the game and store them in "_assets" and "_music". Thus, when you want to display your game's entities and play its sounds, you will only need to use the already loaded content from these two variables.

C. drawGame()

The purpose of the "drawGame" function is to display all entities and play the game's music.

This function takes a pair consisting of an "Entities" and a "Sound" as a parameter.

With "Entities", you obtain the position of the element at index i to be displayed, its color, rotation, size, etc.

With "Sound", you obtain the set of musics to be played.

IV. Conclusion

To conclude, this project is highly modular and allows for the addition of both graphical libraries and games. Do not hesitate to submit PRs to the project to contribute to its development!